Project LEE
Universal Connection Center

Implementation Command for Replit

Goal: make connecting accounts, APIs, local systems, files, webhooks, and future Lamont Labs projects as simple as possible while keeping security, permissions, auditability, and system independence underneath.

Primary Instruction

Create implementation tasks for the work below, then execute those tasks. Do not merely read, summarize, or document this specification.

Build a unified Connection Center inside Project LEE using the existing Systems area, System Contract, Capability Registry, provider abstraction, and Universal Systems API work as the foundation. Do not create a competing integration framework.

Preserve all current integrations, CIL/CerbaSeal boundaries, Android behavior, desktop packaging, data, APIs, and working functionality.

> **Product principle: connecting something to LEE should feel like connecting an app to a phone - simple for the owner, strict and sophisticated underneath.**

1. One Connection Center

Create one owner-facing connection experience under Systems -> Connect / Connections.

The normal UI should let the owner:

- See everything currently connected.
- Add a new account, service, project, API, local system, or data source.
- See Connected / Needs Reauthorization / Degraded / Unavailable / Pending states.
- Review what LEE can read, use, manage, or govern.
- Change permissions without exposing raw credentials.
- Disconnect or reauthorize safely.

2. Supported Connection Methods

Implement the connection architecture so LEE can support these methods through one consistent flow:

Sign in / OAuth - For services that support account authorization. Show Connect, redirect through the provider flow, request only needed scopes, then store the resulting credential securely.

API / Service connection - For independent systems. Accept endpoint plus the required credential/config once, validate it, read the System Contract/capabilities when available, and register the system.

LEE-compatible auto-discovery - For Lamont Labs systems implementing the standard LEE System Contract. LEE should read the manifest, validate compatibility, discover capabilities, show permissions, and ask for owner approval.

Local service connection - Support desktop-era services running on the K6 or approved local network. Build the abstraction now so local endpoints can later be discovered or registered without changing LEE Core.

File / folder source - Allow approved files, folders, repositories, exports, or document sources to be connected/imported without treating them as live APIs.

Webhook / event source - Allow systems to push normalized events into LEE through authenticated, replay-resistant webhook/event endpoints.

Manual credential fallback - Use only when no safer account-auth or managed integration exists.

3. Connection Wizard

For every new connection, use a simple guided flow:

1. Choose what is being connected.
2. Choose the available connection method.
3. Authenticate or provide the required endpoint/credential.
4. Test authentication and reachability.
5. Read manifest/System Contract/capabilities when available.
6. Validate API/contract version and required dependencies.
7. Show requested permissions and authority level in plain language.
8. Let the owner approve or reduce permissions.
9. Run a safe connection test.
10. Register the system and begin health monitoring.

> **Do not make the owner manually reconstruct configuration that LEE can discover from the connected service.**

4. Permissions and Authority

Separate connection from authority. A connected system must not automatically receive broad control.

Use capability-level permissions compatible with:

- OBSERVE - read status, events, metrics, and information.
- USE - call a specialist capability.
- MANAGE - operate explicitly approved controls.
- GOVERNED_MANAGE - consequential operations requiring the declared governance/approval path.

Show these permissions to the owner in plain language. The owner must be able to review and change them later.

5. Secure Credential Handling

The owner-facing UI should never require routine editing of .env files or source code.

Credentials must:

- Use Replit Secrets / approved platform secret storage while LEE is Replit-hosted.
- Use OS-secure credential storage/keychain when the K6 desktop runtime supports it.
- Never be written to normal application tables, model context, Event Log payloads, screenshots, or ordinary logs.
- Support reauthorization and rotation.
- Remain scoped to the minimum capabilities needed.

For Replit-managed AI access, preserve the Replit AI Bridge architecture so desktop LEE does not require separate underlying provider API keys when Replit can supply the model integration.

6. System Contract and Auto-Registration

Use the existing System Contract and Capability Registry as the canonical machine-readable connection contract.

A LEE-compatible system should be able to declare:

- identity and version
- base endpoint / runtime location
- health endpoint
- capabilities and request/response schemas
- events
- permissions
- risk classification
- governance and human-confirmation requirements
- dependencies
- economics/usage metadata where available

> **A future Lamont Labs project should be connectable by implementing this contract, not by adding project-specific logic to LEE Core.**

7. Health, Validation, and Honest Failure

After connection, LEE should automatically know:

- Can I authenticate?
- Is the service reachable?
- Is the contract/API version compatible?
- What capabilities are currently available?
- Are required dependencies healthy?
- Are permissions sufficient?
- Can events be received if supported?

Do not show Connected/Healthy if an advertised capability is unavailable. Surface Needs Reauthorization, Degraded, Unavailable, or Incompatible explicitly.

A failed connection must not silently fall back to a different provider, credential, system, or operation unless that fallback is explicitly part of the declared contract.

8. Connection Types Must Remain Extensible

Do not hardcode the core Connection Center around a fixed list of today's providers.

Provider-specific adapters are allowed where required for OAuth or proprietary APIs, but the core registry, health, permission, capability, audit, and UI model must remain generic.

New adapters or LEE-compatible systems should plug into the existing framework without redesigning the Connection Center.

9. Desktop and Mobile Behavior

Desktop:

- Full Connection Center lives under Systems.
- Support advanced connection details, permissions, diagnostics, and reauthorization.
- Prepare for local/K6 service discovery and OS-secure credential storage.

Android companion:

- Show connection/system status and important reauthorization alerts.
- Allow lightweight approval or reauthorization flows where safe.
- Do not store independent credentials for CIL, CerbaSeal, Replit AI Bridge, Greyline, QuantraCore, or other specialist systems.
- Route specialist-system operations through LEE Core.

10. Auditing and Events

Every connection lifecycle change should generate auditable records/events without exposing secrets.

Track at minimum:

- connection created
- authentication succeeded/failed
- permissions approved/changed
- capabilities discovered/changed
- health changed
- reauthorization required/completed
- connection disabled/disconnected
- contract version changed

11. Required Validation

Create automated tests proving:

1. A mock OAuth-style connection can be registered through the common connection flow.
2. A generic API system can be connected and health-checked.
3. A LEE-compatible System Contract can auto-register capabilities without changing LEE Core.
4. Capability permissions are enforced.
5. A connected system can remain OBSERVE-only.
6. GOVERNED_MANAGE capabilities cannot execute without their declared authorization path.
7. Invalid credentials fail safely.
8. Expired/invalid authorization becomes visible rather than silently ignored.
9. Secrets do not appear in normal logs, events, responses, or UI.
10. Disconnected/degraded systems are shown honestly.
11. Android uses LEE Core rather than direct specialist-system credentials.

12. Existing CIL, CerbaSeal, Replit AI Bridge, Console, Android, API, and desktop build tests continue to pass.

12. Final Deliverable

When complete, report:

- tasks created and completed
- Connection Center routes/screens
- connection methods implemented
- System Contract/Capability Registry changes
- credential-storage design
- permission model
- health and reauthorization behavior
- desktop and Android changes
- tests and build results
- remaining connection types that require future provider-specific adapters

> **Do not claim completion because the screens exist. The connection flows, permissions, secure credential handling, discovery, health validation, and tests must work.**

Final target: CONNECT WITH A SIGN-IN OR ONE CLEAR SETUP FLOW. LET LEE DISCOVER WHAT IT CAN. SHOW PLAIN-LANGUAGE PERMISSIONS. KEEP SECRETS HIDDEN. KEEP COMPLEXITY UNDER THE HOOD.